Software Requirements Specification for Software Engineering: subtitle describing software

Team 6, Blockchainerz
Robert Lam
Nathan Hum
Aidan Lao
Keira Laskoski

October 19, 2026

Contents

1 Introduction **1**
- 1.1 Purpose . . . 2
- 1.2 Scope . . . 2
- 1.3 Stakeholders . . . 3
- 1.4 Definitions and Acronyms . . . 3
- 1.5 Characteristics of Intended Reader . . . 5

2 System Overview **5**
- 2.1 Problem Statement . . . 5
- 2.2 Product Vision . . . 6
- 2.3 System Context . . . 7
  - 2.3.1 System Context Diagram . . . 7
  - 2.3.2 Responsibilities . . . 7
  - 2.3.3 Operating Context and Safety Significance . . . 8
- 2.4 User Characteristics . . . 8
- 2.5 System Constraints . . . 9

3 Stakeholder Requirements **9**

4 User Personas **11**
- 4.1 Persona 1: Emergency Management Coordinator (Task Requester) . . . 11
- 4.2 Persona 2: Independent Stock Forecaster (Inference Provider) . . . 11
- 4.3 Persona 3: Hydrology Researcher (Inference Provider) . . . 12

5 Use Cases **12**
- 5.1 UC1: Example Use Case . . . 12

6 Functional Requirements **13**
- 6.1 Accounts and Authorization . . . 13
- 6.2 Inference Tasks . . . 13
- 6.3 Provider Registration and Staking . . . 14
- 6.4 Inference Requests . . . 14
- 6.5 Provider Acceptance and Predictions . . . 15
- 6.6 Aggregation and Results . . . 16
- 6.7 Reputation and Rewards . . . 16
- 6.8 Penalties . . . 17
- 6.9 Transparency . . . 17
- 6.10 Data Handling . . . 18

7 Data Requirements **18**
7.1 Major Data Entities . . . 18
7.2 Data Dictionary . . . 19
7.3 Privacy and Data Retention . . . 20

8 Non-Functional Requirements **20**

9 AI and Machine Learning Requirements **21**
9.1 Evaluation Metrics . . . 22
9.2 Known Limitations . . . 22

10 Requirements Traceability Matrices **22**

11 Rationale **22**
11.1 Provider Staking . . . 23
11.2 Reputation Tolerance . . . 23

12 Likely Changes **24**

13 Unlikely Changes **24**

14 Appendices **25**
14.1 Appendix A: Additional Materials . . . 25
14.2 Appendix B: Development Plan . . . 25
14.3 Appendix C: Values of Auxiliary Constants . . . 25
14.4 Appendix D: Generative AI Disclosure . . . 25
14.5 Appendix E: Reflection . . . 26

Revision History

Date	Version	Notes
October 8	1.0	Aidan: Added sections 1.1 to 1.5 (Purpose, Scope, Stakeholders, Definitions and Acronyms, and Characteristics of Intended Reader)
October 11	1.0	Robert: Added major functional requirements
October 11	1.0	Aidan: Grouped functional requirements into categories and added functional requirements derived from the stakeholder requirements
October 11	1.0	Nathan: Added all of section 2, likely changes and unlikely changes
October 16	1.0	Robert: Added data and AI reqs sections

[All capstone SRS documents should have a reflection appendix. —TPLT]

[This SRS template is based on the IEEE template, with input from co-pilot and S. Smith. It will get you started. You should not modify the section headings, without first discussing the change with the course instructor. Modification means you are not following the template, which loses some of the advantage of a template, especially standardization. —TPLT]

[Feel free to change the appearance of the report by modifying the LaTeX commands. —TPLT]

[This template document assumes that a single program is being documented. If you are documenting a family of models, you should start with a commonality analysis. —TPLT]

[The SRS is not generally written, or read, sequentially. The SRS is a reference document. It is generally read in an ad hoc order, as the need arises. —TPLT]

[Guiding principles for the SRS document:

- Do not repeat the same information at the same abstraction level. If information is repeated, the repetition should be at a different abstraction level.

—TPLT]

[The template description comments should be disabled before submitting this document for grading. —TPLT]

[You can borrow any wording from the text given in the template. It is part of the template, and not considered an instance of academic integrity. Of course, you need to cite the source of the template. —TPLT]

[When the documentation is done, it should be possible to trace back to the source of every piece of information. Some information will come from external sources, like terminology. —TPLT]

[An SRS document should have the following qualities: unambiguous, consistent, complete, validatable, abstract and traceable. —TPLT]

[The overall goal of the SRS is that someone can learn, understand and verify the captured domain knowledge. They should not have to trust the authors of the SRS on any statements. They should be able to independently verify/derive every statement made. —TPLT]

1 Introduction

[Explain what system is being specified. State purpose, users, and project goals. —TPLT]

[Introduce the name of your software as Software Engineering. Using Software Engineering as a command lets you change the name if necessary and have the change propagate to all documents. —TPLT]

[Include a reference to the template this work is based on. **?** justifies the SRS structure and **?** provides the broader life-cycle context that underlines modern requirements engineering practice. —TPLT]

1.1 Purpose

DAIIM (Decentralized Artificial Intelligence Inference Marketplace) exists to reduce reliance on the small number of large organizations that currently provide most AI inference. It serves two groups of people.

For people who need machine learning predictions, DAIIM provides a way to obtain inferences for a task they define without trusting a single centralized provider. Each inference request is answered by multiple independent providers, and their predictions are combined into one consensus result. Because no single provider controls the outcome, the result is more reliable, harder to censor or manipulate, and not dependent on any one point of failure.

For people who build machine learning models, DAIIM provides an open marketplace where they can sell predictions from their models and earn money for accurate results. This gives independent providers a way to compete with large organizations and a reason to keep improving their models.

1.2 Scope

DAIIM allows task requesters to obtain reliable inferences by combining predictions from multiple independent inference providers.

The inputs to the system consist of task definitions, inference requests, and network participation. Task definitions are specifications of a problem to be solved via machine learning, including the expected input and output types. Inference requests ask the network to perform inference for a given task and include input data or some specification of where to find the input data. Network participation includes machine learning predictions for individual inference requests that are supplied to the network by an individual. The system outputs aggregated inference results and maintains information about the network. The aggregated inference results include the final prediction for a given inference request, as well as information about the quality of the prediction.

The expected benefits are reduced reliance on centralized AI providers, improved reliability of inferences through redundant predictions from independent providers, and incentives for providers to improve the quality, performance, availability, and cost-effectiveness of their models.

The following are outside the scope of DAIIM:

- Inference tasks whose output is not a single numeric value, such as predicting a category label. Support for additional model types is a stretch goal.
- Inference on private or sensitive data within Trusted Execution Environments (TEEs). This is a stretch goal.
- DAIIM doesn't train models. It just uses model providers' existing models to make predictions for inference requests.
- Using a custom blockchain. DAIIM will use Solana, an existing public blockchain.

1.3 Stakeholders

Primary Stakeholders

Primary stakeholders interact with DAIIM directly.

- **Task requesters**
  Task requesters are people who are requesting inferences to be made for a given task, using their own input data. These people value consensus and reliability: they want a prediction that combines the results of multiple independent models, rather than relying on a single model that may be wrong, biased, or unavailable. For example, emergency management agencies would make requests to predict river levels or wildfire risk, where a single incorrect forecast can leave a community unprepared. Similarly, decentralized finance platforms could make requests to predict stock prices, where relying on a single source of predictions creates a risk of error or manipulation. Because input data is shared with inference providers, task requesters are expected to submit data that is not sensitive.

- **Inference providers**
  Inference providers are people with computing resources and models, and want to monetize their regression models. These people usually have specialized or powerful models that can be used to make predictions for a given task.

Secondary Stakeholders

Secondary stakeholders do not use DAIIM directly, but are affected by the inferences it produces or by how it handles data.

- **Beneficiaries of machine learning inference**
  Beneficiaries are people and organizations, such as laboratories, that rely on the results of machine learning inference without requesting inferences themselves. For example, a laboratory may make decisions based on biomarker predictions that a healthcare researcher obtained through DAIIM. These people benefit from inferences that are reliable and not controlled by a single provider.

- **Regulatory bodies**
  Regulatory bodies are organizations that oversee machine learning inference and data privacy. They want to ensure that personal information and sensitive data handled by DAIIM, such as inference inputs and training data, is only accessible to the parties that need it, is encrypted, and is not retained longer than necessary to fulfill an inference request.

1.4 Definitions and Acronyms

Term	Definition
AI	Artificial Intelligence.
ML	Machine Learning.
DAIIM	Decentralized Artificial Intelligence Inference Marketplace, the system specified in this document.
Task	A problem to be solved via machine learning, such as predicting a biomarker level or a stock price.
Task definition	A specification of a task, including the expected input and output types.
Inference	A prediction produced by a machine learning model for a given input.
Inference request	A request for the network to perform inference for a given task, including input data or a specification of where to find the input data.
Task requester	A person who requests inferences to be made for a given task.
Inference provider	A person with computing resources and models who supplies predictions for inference requests to the network.
Network participation	Machine learning predictions for individual inference requests supplied to the network by an inference provider.
Aggregated inference result	The final prediction for a given inference request produced by combining the predictions of multiple inference providers, together with information about the quality of the prediction.
Ground truth	The true, observed value that a prediction is attempting to estimate.
Linear regression	A machine learning model that predicts a numeric output as a linear combination of its input features.
Decentralized	Operating without a single central organization that coordinates or controls the system.
Blockchain	A distributed, append-only ledger of transactions that is maintained by a network of nodes rather than a central authority.
On-chain	Recorded on, or executed by, a blockchain.
Web3	Applications built on blockchains, where users interact with the system through their own blockchain accounts and authorize actions by signing transactions.
Token	A digital asset on a blockchain, used by DAIIM to compensate inference providers.

TEE	Trusted Execution Environment. A secure area of a processor that protects the code and data loaded inside it from unauthorized access.

Table 1: Definitions and Acronyms

1.5 Characteristics of Intended Reader

The intended readers of this document are the software engineers, reviewers, and supervisors who will design, implement, verify, and maintain DAIIM. Readers are expected to have:

- An undergraduate-level background in software engineering, including requirements specification, software design, and testing, equivalent to the third year of a software engineering degree.
- Introductory knowledge of machine learning, including supervised learning, linear regression, model training versus inference, and basic error measures such as relative error, equivalent to an introductory undergraduate machine learning course.
- Introductory knowledge of blockchain and Web3 concepts, including blockchain transactions, accounts and wallets, transaction signing, and tokens. Familiarity with smart contract development is helpful but not required.
- Familiarity with web applications that run in a modern browser.

Terms specific to the domain of DAIIM are defined in Table 1.

2 System Overview

This section provides general information about the system. It identifies the problem, the vision, and the interfaces between the system and its environment. This section also describes user characteristics and lists the system constraints.

2.1 Problem Statement

AI inference services often rely on centralized providers, which can create single points of failure and leave users dependent on a single service for their results. Users may also have limited visibility into how results are obtained, how reliable they are, and whether the service provider is operating as expected. These limitations raise concerns about service availability, trust, and the ability to assess the quality of inference results. Relying on a single provider also limits opportunities for independent providers to contribute their inference capabilities to a shared marketplace. DAIIM addresses the need for an approach to AI inference that supports greater reliability, accountability, and participation. The underlying problem is how to provide users with confidence in inference results while allowing multiple providers to

participate and be evaluated fairly, without requiring users to rely entirely on one centralized service.

2.2 Product Vision

DAIIM aims to make AI inference more reliable, transparent, and accessible by connecting users who need predictions with independent providers who can supply them. By combining predictions from multiple providers and making relevant performance and transaction information available, the marketplace seeks to build trust in inference results while creating opportunities for providers to offer their services. The system's goals are to:

- Improve the reliability of inference results by combining predictions from multiple independent providers, assessing the quality of their contributions, and reducing dependence on any single provider.
- Create a fair and accountable marketplace in which providers can build reputations and receive compensation based on their performance.
- Make marketplace activities transparent by allowing users to inspect aggregated results, relevant provider performance information, and records of transactions and payments.
- Make AI inference accessible to users without requiring prior knowledge of AI or Web3 technologies, while giving inference providers a way to offer their services and earn compensation for their contributions.

2.3 System Context

2.3.1 System Context Diagram

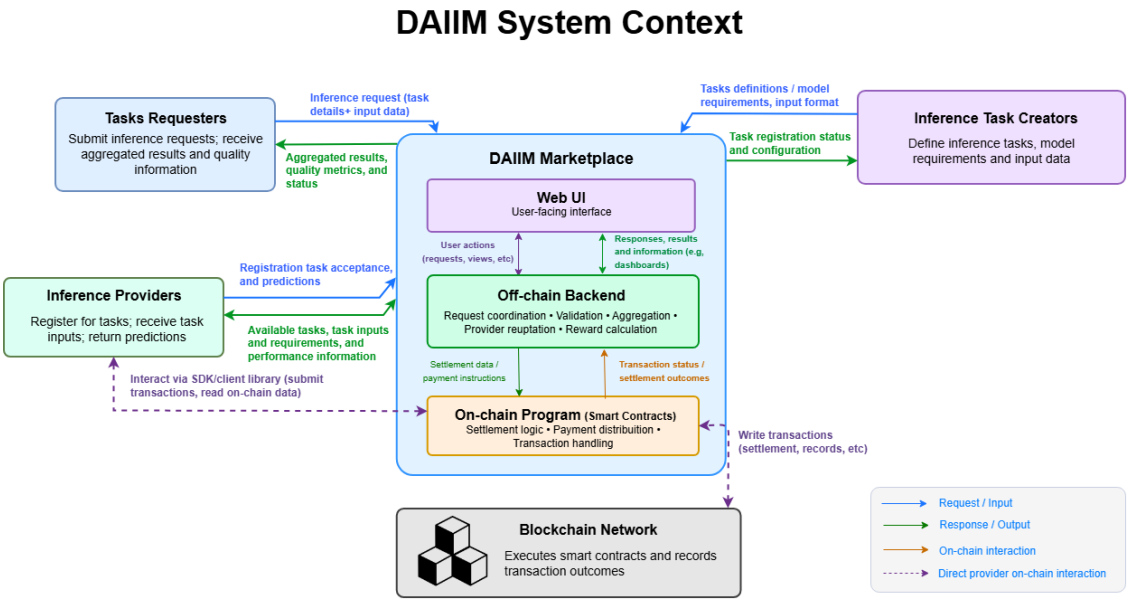


Figure 1: System Context

2.3.2 Responsibilities

- **Task Requester Responsibilities:**
  - Define the inference task by specifying the problem to be solved, the expected input and output formats, and other required task settings.
  - Submit requests for predictions using an existing task definition and input data that conforms to the task's requirements.
  - Specify the amount of tokens offered for an inference request and authorize the associated payment transaction.
  - Receive the final aggregated prediction, information about the range of provider predictions, the number of providers that submitted predictions, and the request status.
  - View the status and results of previously submitted inference requests.
- **Inference Provider Responsibilities:**
  - Register to provide predictions for an inference task by staking the required amount of tokens.

- Receive information about available requests for tasks they are registered for, including the task requirements, input data, proposed payment, and relevant timing information.
- Accept or decline participation in available inference requests.
- Submit a prediction that conforms to the task's required output format for each request they accept.
- Access information about their reputation, staked tokens, and rewards earned for each registered task.

- **Blockchain Network Responsibilities:**
  - Process marketplace transactions authorized by users, including transactions related to request payments, provider stakes, and provider compensation.
  - Record marketplace information designated for on-chain storage, including request payments, provider stakes, aggregated results, reputation updates, and reward distributions.
  - Make transaction status and transaction records available for verification.

2.3.3 Operating Context and Safety Significance

DAIIM is intended to be used as a web-based marketplace for requesting and evaluating AI inference results from multiple independent providers. Users submit inference requests through the system and receive aggregated predictions along with information about the contributing providers and the results they submitted. Blockchain technology is a required component of DAIIM, supporting transparent and verifiable records of marketplace activities, including provider stakes, payments, aggregated results, and reward distributions. DAIIM may be used in applications where reliable predictions are important, such as wildfire detection, where inaccurate or unavailable results could delay the identification of potential hazards. By combining predictions from multiple providers, the system aims to reduce dependence on any single provider and improve the reliability and availability of inference results. However, using multiple providers does not guarantee a correct result, as providers may submit inaccurate predictions or share similar sources of error. Although DAIIM does not directly control physical systems or make decisions on behalf of its users, its results may inform safety-critical applications where incorrect predictions could have serious consequences. Users are therefore responsible for validating aggregated results and determining whether they are sufficiently reliable for their intended use. The system must also account for risks such as provider collusion, unavailable providers, failed transactions, and incorrect aggregation or reputation calculations, as these may affect the trustworthiness of its results and marketplace operations.

2.4 User Characteristics

DAIIM is intended to support users with varying levels of technical expertise.

- **Task Requesters:** Task requesters should have a basic understanding of the inference task they wish to perform and be able to provide the required input data. They should be able to use a modern web browser and interpret the inference results presented by the system. They are not expected to understand the internal aggregation, reputation, or reward algorithms.
- **Inference Providers:** Inference providers should have sufficient technical knowledge and computational resources to execute the inference tasks they accept. They should understand the requirements for submitting predictions in a format supported by the marketplace.

The user interface should minimize the technical knowledge required to submit requests and interpret results, while provider participation may require greater technical expertise.

2.5 System Constraints

- **Web3 browser compatibility:** The user-facing application must be accessed through a web browser that supports the Web3 wallet extensions required to authenticate users and authorize blockchain transactions.
- **Inference type and data compatibility:** The system is restricted to regression-based inference for the initial project. Inference requests must provide input data in a supported format, and provider predictions must conform to the expected numerical output format so that results can be validated and aggregated consistently.
- **Blockchain integration:** Marketplace operations that require on-chain execution must be compatible with the blockchain network selected for the project. The system must account for possible transaction delays or failures.

3 Stakeholder Requirements

This section lists the stakeholder requirements for DAIIM, grouped by the stakeholders described in Section 1.3. These requirements are technology-independent and are stated from the point of view of the stakeholders.

Task Requesters

SR1 Task requesters shall be able to define their own machine learning task, including the type of input the task takes and the type of output it produces.

SR2 Task requesters shall be able to submit an inference request for a defined task and receive a result for that request.

SR3 Task requesters shall receive inference results that are reliable. Each result shall be based on predictions from multiple independent inference providers, so that no single faulty or dishonest provider can determine the result.

SR4 Task requesters shall receive results for their inference requests even when some inference providers are unavailable.

SR5 Task requesters shall receive information about the quality of each inference result, so that they can judge how much to trust it.

SR6 Task requesters shall be able to obtain inferences without relying on a single organization to coordinate the request, supply the prediction, or control the result.

SR7 Task requesters shall be able to see the actions and payments they authorize, and verify that those actions and payments were carried out.

SR8 Task requesters shall be able to define tasks, submit inference requests, and receive results without knowledge of decentralized systems or their underlying infrastructure.

Inference Providers

SR9 Inference providers shall be able to offer predictions from their own models for tasks defined by task requesters.

SR10 Inference providers shall be able to earn a profit by supplying accurate predictions to the network.

SR11 Inference providers shall be rewarded according to the accuracy of their predictions, so that supplying honest, accurate predictions is more profitable than supplying dishonest or careless ones.

SR12 Inference providers shall receive payment for their predictions without manual intervention from the task requester or any third party.

Secondary Stakeholders

SR13 Personal information and sensitive data handled by DAIIM, such as inference inputs and training data, shall only be accessible to the parties that need it to fulfill an inference request.

SR14 Personal information and sensitive data handled by DAIIM shall be protected from unauthorized access while it is transferred and while it is stored.

SR15 Personal information and sensitive data handled by DAIIM shall not be retained longer than necessary to fulfill the inference request it belongs to.

4 User Personas

4.1 Persona 1: Emergency Management Coordinator (Task Requester)

- Name: Priya Nair
- Description: Priya is 41 and coordinates emergency management for a mid-sized town on a flood-prone river, in a region that also has an annual wildfire season. She decides when to issue flood warnings and evacuation orders, and when to pre-position firefighting resources. She currently relies on a single forecasting service, and has to justify every decision to the town council.
- Goals: Receive river-level forecasts 24 to 48 hours ahead, and wildfire risk estimates during fire season, that combine predictions from multiple independent sources. See how much the sources agree, so she knows how much to trust each forecast. Keep costs predictable and within the town's budget.
- Frustrations: Her current service is a black box: it gives one number with no indication of confidence, and she has no alternative when it is wrong or unavailable. A missed flood forecast the previous spring left the town unprepared. She has no interest in learning about cryptocurrency or managing digital wallets.
- Technical Ability: Comfortable with spreadsheets, web dashboards, and mapping tools. No programming experience and no experience with blockchain or Web3 applications.

4.2 Persona 2: Independent Stock Forecaster (Inference Provider)

- Name: Daniel Okafor
- Description: Daniel is 29 and works as a software developer. In his spare time he builds machine learning models that predict next-day stock prices from historical market data. His models perform well in backtests, but he has no way to earn money from them other than trading with his own savings.
- Goals: Earn a steady side income by supplying predictions from his models. Get paid automatically and promptly for good predictions. Build a visible track record that shows his models are reliable. Keep the cost of submitting each prediction low enough that participating is worthwhile.
- Frustrations: Selling predictions usually requires connections at a fund. Trading his own money exposes him to large losses. Prediction platforms he has tried were unclear about how payouts were calculated.
- Technical Ability: Strong Python programmer who regularly uses machine learning libraries. Owns a cryptocurrency wallet and has used decentralized finance applications.

4.3 Persona 3: Hydrology Researcher (Inference Provider)

- Name: Dr. Elena Varga
- Description: Elena is 34 and is a postdoctoral researcher in hydrology. She specializes in forecasting streamflow in mountain rivers fed by melting snow, where standard forecasting models often perform poorly during the spring melt. Her models currently only appear in academic papers.
- Goals: Earn supplementary income from her expertise by supplying predictions for tasks in her field, such as river-level and streamflow forecasts. Build a reputation for her models outside academia. See her research help communities prepare for floods.
- Frustrations: Consulting contracts are slow to arrange and rarely available to early-career researchers. Her models are most valuable during unusual events, when they disagree with standard models, and she worries about being penalized for disagreeing with the majority even when she is right.
- Technical Ability: Expert in statistical modeling, with strong Python and R skills. Has never used a cryptocurrency wallet or a blockchain application.

5 Use Cases

[Use cases are a way to describe the interactions between users and the system. They are typically written in natural language and describe the steps that a user takes to achieve a specific goal. Use cases can be used to identify functional requirements and to create test cases. —TPLT] [Typically 8-15 substantial use cases. —TPLT]

5.1 UC1: Example Use Case

Primary Actor

TODO

Preconditions

- TODO

Main Success Scenario

1. TODO
2. TODO
3. TODO

Alternative Flows

- TODO

Postconditions

- TODO

6 Functional Requirements

[Functional requirements describe what the system should do. They are typically expressed as a set of conditions or capabilities that the system must have. These requirements are usually derived from the stakeholder requirements and use cases. —TPLT]

[Use SHALL statements. Requirements must be atomic, testable, and traceable. —TPLT]

The functional requirements are grouped by the area of DAIIM they describe.

6.1 Accounts and Authorization

FR1: The system shall allow users to authenticate by connecting their Web3 wallet and signing a verification message to prove ownership of the associated wallet address.

FR2: The system shall allow unauthenticated users to browse publicly available marketplace information, including registered providers, available models, and provider reputations.

FR3: The system shall require authenticated users to authorize (sign) blockchain transactions through their connected wallet before executing actions that transfer tokens or modify on-chain state.

6.2 Inference Tasks

FR4: The system shall allow authenticated users to create new inference tasks by providing the task definition.

FR5: Each inference task definition shall specify a name, problem description, inference category or type, input schema, and output schema. Where applicable, it shall also specify the metric(s) used to evaluate predictions. The task creator may optionally provide training data, or a reference to training data, for providers to use when training or adapting models for the task.

FR6: Each inference task definition shall specify the minimum number of participating providers required to fulfill an inference request for the task.

FR7: The system shall reject inference task definitions that specify a minimum number of participating providers less than 2.

FR8: Each inference task definition shall specify a reputation tolerance T, in the units of the task's output, used to determine when a provider's reputation decreases.

FR9: Each inference task definition shall specify the length of the acceptance period for the task's inference requests. The acceptance period for each request shall end this length of time after the request is broadcast.

FR10: Each inference task definition shall specify the maximum prediction time for the task's inference requests, measured from the end of the acceptance period.

FR11: The system shall allow all users to browse existing inference tasks and view each task's definition, required minimum provider stake, and number of registered providers.

6.3 Provider Registration and Staking

FR12: The system shall allow authenticated users to register as inference providers for an inference task by staking the required minimum amount of tokens (reference the min # tokens).

FR13: When a provider registers for an inference task, the system shall lock the required minimum stake for that task in escrow. The system shall return the provider's stake when the applicable conditions for releasing it are met.

FR14: The system shall allow registered inference providers to unregister from an inference task, provided they have not accepted any acitve inference requests for that task. Upon unregistration, the provider shall be removed from the list of registered providers for the task, and their remaining staked tokens shall be returned to their token account. Upon re-registering for an inference task, the provider shall start with the baseline reputation score for that task, regardless of their previous reputation score.

FR15: The system shall allow inference providers to view their reputation score, staked tokens, and rewards earned for each inference task they are registered for.

6.4 Inference Requests

FR16: The system shall allow authenticated users to submit inference requests for an existing inference task. Each request shall include input data that conforms to the task's input schema.

FR17: The system shall reject inference requests whose input data does not conform to the task's input schema.

FR18: When submitting an inference request, the system shall require the requester to choose a payment amount in tokens greater than the minimum and prompt them to authorize, through their connected wallet, a transaction transferring that amount of tokens into escrow.

FR19: After an inference request is submitted, the system shall broadcast its task definition, proposed payment amount, acceptance-period expiry time, and maximum prediction time to all providers registered for the task.

6.5 Provider Acceptance and Predictions

FR20: Upon broadcast of an inference request, the system shall automatically begin the acceptance period, during which providers registered for the task may accept or decline participation.

FR21: At the specified expiry time, the system shall end the acceptance period. All registered providers that have not accepted the request will be treated as non-participating. Inference request acceptances from inference providers outside of the acceptance period will not be considered.

FR22: If fewer than the task's minimum number of participating providers have accepted an inference request when the acceptance period ends, the system shall cancel the request and return the requester's escrowed payment.

FR23: The system shall make the request's input data, or information needed to access it, available to all providers that accepted the request during the acceptance period.

FR24: Upon expiration of the acceptance period, the system shall begin collecting inferences from the set of providers that accepted the request. Predictions returned from inference providers that are not participating will be rejected.

FR25: The system shall allow each provider that accepted an inference request to submit a prediction for that request during the prediction phase.

FR26: The system shall accept at most one prediction from each provider for a given inference request.

FR27: The system shall reject predictions that do not conform to the task's output schema.

FR28: The system shall end the prediction phase when it has received a prediction from every provider that accepted the request or when the maximum prediction time has elapsed, whichever occurs first. The system shall not accept predictions after the phase has ended.

FR29: If fewer than the task's minimum number of participating providers have submitted predictions when the prediction phase ends, the system shall cancel the request and return the requester's escrowed payment.

6.6 Aggregation and Results

FR30: Once the prediction phase ends, the system shall aggregate the predictions into a final numerical result using a weighted aggregation algorithm in which each provider's reputation determines the weight assigned to its prediction.

FR31: The system shall return the final aggregated result, time elapsed since submitting the request and the number of inference providers that submitted predictions, to the user that submitted the inference request.

FR32: The system shall return, with each final aggregated result, the minimum and maximum of the predictions submitted for the request.

FR33: The system shall allow requesters to view the status of each inference request they have submitted and, once available, its final aggregated result.

6.7 Reputation and Rewards

FR34: After the final aggregated result for an inference request is computed, the system shall update the reputation score of each provider that submitted a prediction for the request. For each such provider i, the system shall compute d_i, the absolute difference between the provider's prediction and the final aggregated result, and $\bar{d}$, the average of d_j over all providers that submitted predictions for the request.

FR35: The system shall increase the reputation score of a provider whose distance d_i is less than or equal to the average distance $\bar{d}$.

FR36: The system shall leave unchanged the reputation score of a provider whose distance d_i is greater than $\bar{d}$ but less than or equal to $\bar{d} + T$, where T is the task's reputation tolerance.

FR37: The system shall decrease the reputation score of a provider whose distance d_i is greater than $\bar{d} + T$, where T is the task's reputation tolerance.

FR38: The system shall limit the reputation decrease a provider receives for a single inference request to at most MAX_REPUTATION_PENALTY, regardless of how far the provider's prediction is from the final aggregated result.

FR39: The system shall distribute the inference request's reward among all providers whose predictions contributed to the final aggregated result and whose reputation did not decrease as a result of the request, in proportion to each provider's reputation score.

6.8 Penalties

FR40: The system shall slash a provider's stake only under the conditions specified in FR41 and FR42, and shall burn a provider's remaining stake only under the conditions specified in FR44 and FR45.

FR41: When the system rejects a provider's prediction under FR27, the system shall slash SLASH_FRACTION of the provider's stake for the task and decrease the provider's reputation score.

FR42: When a provider that accepted an inference request has not submitted any prediction for the request by the end of the prediction phase, the system shall slash SLASH_FRACTION of the provider's stake for the task and decrease the provider's reputation score.

FR43: Whenever a provider's stake for an inference task falls below MINIMUM_PROVIDER_STAKE, the system shall automatically unregister the provider from that task.

FR44: When a provider has been slashed for at least REPEAT_OFFENSE_COUNT of the last REPEAT_OFFENSE_WINDOW inference requests they accepted for a task, the system shall unregister the provider from the task and burn their remaining stake for the task.

FR45: The system shall automatically unregister any inference provider whose reputation score falls below the minimum reputation threshold. Upon removal, the provider's remaining staked tokens for that inference task shall be burned.

6.9 Transparency

FR46: Before prompting a user to authorize a transaction, the system shall display the action being performed and the amount of tokens being transferred.

FR47: The system shall display the status of each transaction a user has authorized until the transaction is confirmed or fails.

FR48: The system shall provide, for each transaction a user has authorized, the transaction identifier needed to look up the transaction on the blockchain.

FR49: The system shall record request payments, provider stakes, aggregated results, reputation updates, and reward distributions on the blockchain.

6.10 Data Handling

FR50: The system shall make a request's input data available only to the requester and the providers that accepted the request.

FR51: The system shall not store a request's input data on the blockchain in readable form.

FR52: The system shall delete a request's input data from any storage the system controls once the request has been completed or cancelled.

Questions / Missing reqs:
- We said that the task requester gets to choose the amount of tokens they want to offer for a request - How can we express this to inference providers in a useful way? Since the total amount does not reflect how much they will earn.
- Unless the requester also gets to choose exactly how many participants are necessary to complete the request.
- What happens if aggregation fails? Should add req for this
- What happens if not enough providers accept? Should add req for this
- Rewards distribution and provider reputation update may be changed after POC
- Need to define some of the thresholds used in reqs above: min reputation, amount to stake, min inference request payment etc.

7 Data Requirements

7.1 Major Data Entities

- Model Providers: Network contributors identified by a wallet address, with a list of registered inference tasks and a reputation for each task.
- Inference Tasks: A task defined by its description and input and output formats, with a list of registered inference providers.
- Inference Requests: A consumer's request for an inference task, including associated inference task, input data, cost, information about the requester, individual provider inferences and the final aggregated inference.
- Token Economics: Network-wide totals describing token supply, staking, escrow, burns, and reward distributions.
- Network Participation Metrics: Measurements summarizing model providers' participation in inference tasks and requests across the network.

7.2 Data Dictionary

[For each entity, define all important attributes. —TPLT]

Entity	Attribute	Type	Description
Model Provider	Wallet address	Address	Provider's wallet
	Registered tasks	Task IDs	Tasks the provider joined
	Task reputation	Number	Reputation for a task
	Task stake	Integer	Stake for a task
Inference Task	Task ID	Identifier	Unique task identifier
	Name and type	String	Task name and category
	Description	String	Problem addressed by the task
	Input schema	Json	Valid request input format
	Output schema	Json	Valid prediction format
	Minimum providers	Integer	Required participants per request
	Timing limits	Timestamp	Acceptance and prediction periods
	Reputation tolerance	Number	Threshold for reputation changes
	Minimum stake	Number	Stake required for registration
Inference Request	Request ID	Identifier	Unique request identifier
	Task ID	Identifier	Requested inference task
	Requester wallet	Address	Wallet of the requester
	Input data	Json	Data submitted for inference
	Cost	Number	Offered payment
	Status	Status	Current request state
	Provider predictions	List¡Output Type¿	Predictions submitted for the request
	Aggregated result	Output Type	Final inference result
	Submission time	Timestamp	Request creation time
Token Economics	Total minted	Number of tokens	Tokens minted across the network
	Total staked	Number of tokens	Tokens staked by providers
	Request escrow	Number of tokens	Tokens escrowed for inference requests
	Total burned	Number of tokens	Tokens permanently removed from supply

Rewards distributed	Number of tokens	Tokens paid to providers as rewards

Table 2: Data Dictionary

7.3 Privacy and Data Retention

DAIIM will use a semi-private approach for inference request input data. Inputs will be encrypted and made accessible only to the requester and providers that accept the request. By submitting a request, the requester consents to the input being shared with those providers and should not submit data they consider fully private or confidential. Input data will be stored only as long as needed to fulfill the request and will be deleted from storage controlled by the system once the request is completed or cancelled, following established data-minimization and retention practices. The system will not store readable input data on the blockchain. DAIIM's handling of personal data must follow applicable privacy and data-protection laws for its deployment and users; the specific obligations will depend on the relevant jurisdictions.

Other data, including wallet addresses, provider reputations, task definitions, request status and results, and token-economics records, will be collected and retained only as needed to operate the marketplace, provide its features, and meet applicable legal obligations. Information needed for public marketplace and blockchain functions may be visible to other users; users should be informed that blockchain records may be permanent and cannot necessarily be deleted. Data stored off-chain will be protected against unauthorized access and encrypted in transit and at rest where appropriate.

8 Non-Functional Requirements

[You may have to add other NFRs. —TPLT]

[List your nonfunctional requirements. You may consider using a fit criterion to make them verifiable. —TPLT]

[The goal is for the nonfunctional requirements to be unambiguous, abstract and verifiable. This isn't easy to show succinctly, so a good strategy may be to give a "high level" view of the requirement, but allow for the details to be covered in the Verification and Validation document. —TPLT]

[An absolute requirement on a quality of the system is rarely needed. For instance, an accuracy of 0.0101 % is likely fine, even if the requirement is for 0.01 % accuracy. Therefore, the emphasis will often be more on describing now well the quality is achieved, through experimentation, and possibly theory, rather than meeting some bar that was defined a priori. —TPLT]

[You do not need an entry for correctness in your NFRs. The purpose of the SRS is to record the requirements that need to be satisfied for correctness. Any statement of correctness would just be redundant. Rather than discuss correctness, you can characterize

how far away from the correct (true) solution you are allowed to be. This is discussed under accuracy. —TPLT]

NFR1: **Accuracy** [Characterize the accuracy by giving the context/use for the software. Maybe something like, "The accuracy of the computed solutions should meet the level needed for <engineering or scientific application>. The level of accuracy achieved by Software Engineering shall be described following the procedure given in Section X of the Verification and Validation Plan." If you do this provide a link to the VnV plan. —TPLT]

NFR2: **Usability** Users with no knowledge of decentralized systems or their underlying infrastructure shall be able to define tasks, submit inference requests, and receive results without assistance. In usability testing with participants who have no prior blockchain or Web3 experience, at least USABILITY_COMPLETION_RATE of participants shall complete each of these tasks without assistance.

NFR3: **Security** Personal information and sensitive data handled by DAIIM, such as inference inputs, shall be encrypted while it is transferred and while it is stored by the system, so that it is protected from unauthorized access.

NFR4: **Maintainability** [The effort required to make any of the likely changes listed for Software Engineering should be less than FRACTION of the original development time. FRACTION is then a symbolic constant that can be defined at the end of the report. —TPLT]

NFR5: **Portability** [This NFR is easier to write than the others. The systems that Software Engineering should run on should be listed here. When possible the specific versions of the potential operating environments should be given. To make the NFR verifiable a statement could be made that the tests from a given section of the VnV plan can be successfully run on all of the possible operating environments. —TPLT]

- Other NFRs that might be discussed include verifiability, understandability, reusability, performance, reliability and availability, security, and accessibility. You may have to add other NFRs.

9 AI and Machine Learning Requirements

This section describes how DAIIM supports machine-learning inference, evaluates prediction quality, and communicates the limitations of its results. The project scope focuses on regression tasks: each task defines its input and output schemas, providers submit numerical predictions, and the system combines those predictions into a reputation-weighted result.

9.1 Evaluation Metrics

Evaluation will focus on accuracy score, as this is appropriate for regression tasks. Ground truth may not exist or may become available only later; when it does become available, the corresponding accuracy metrics will be provided and displayed in the system.

9.2 Known Limitations

The system aggregates provider predictions but does not independently verify that the final result is correct. Decentralization, provider reputation, and staking are intended to encourage reliable predictions, but the system cannot guarantee that a result is accurate or suitable for its intended use.

10 Requirements Traceability Matrices

[The Requirements Traceability Matrix (RTM) is a table that shows the relationship between different types of requirements. It is used to ensure that all requirements are traceable and that no requirements are missed. The typical flow is: Stakeholder Need -¿ Use Case -¿ Requirement —TPLT]

[There is also a traceability matrix between functional and nonfunctional requirements. The purpose of some FRs is to satisfy NFRs. Therefore, it is important to show the relationship between the two. —TPLT]

[Further traceability to test cases and modules will be given in other documents. —TPLT]

Stakeholder Req	Use Case	Functional Req
SR??	UC1	FR1

Table 3: Traceability Between Stakeholder Requirements, Use Cases, and Functional Requirements

Functional Req	Nonfunctional Req
FR??	NFR1

Table 4: Traceability Between Functional and Nonfunctional Requirements

11 Rationale

[Provide a rationale for the decisions made in the documentation. Rationale should be provided for scope decisions, functional requirements, nonfunctioanl requirements, data re-

quirements, and AI requirements. You don't need a rationale for everything, ust the major decisions that are not obvious. You should have traceability to the entities in the document that you are referencing. —TPLT]

11.1 Provider Staking

Inference providers must stake a minimum amount of tokens to register for an inference task (FR12). The stake is locked in escrow by the system's on-chain logic, rather than held by any person, and can only be returned, slashed, or burned according to the rules in FR13, FR40, FR14, and FR45. Requiring a stake serves three purposes:

- **Resistance to fake accounts.** The final result is a reputation-weighted aggregate of provider predictions (FR30), so anyone who controls enough providers for a task can control its results. Without a stake, an attacker could register many providers at no cost, and a provider with a low reputation could abandon their account and register a new one. Requiring a stake for every registration makes each additional provider cost real tokens, which makes gaining control of a task's results expensive.
- **Penalties for misbehaviour.** A loss of reputation alone may not deter a provider who has nothing else at risk. A stake allows the system to impose a financial penalty by slashing it (FR40), or burning it when a provider's reputation falls below the minimum threshold (FR45).
- **Commitment to accepted requests.** If accepting an inference request had no cost, providers could accept every request and respond only when convenient, causing requests to be cancelled for having too few predictions (FR29). Having a stake at risk gives providers a reason to submit a prediction for every request they accept.

A provider's stake does not determine the weight of their predictions; reputation does (FR30). This prevents providers from gaining influence over results through wealth alone, so influence must be earned through a history of predictions that agree with the network.

The minimum stake must balance two concerns. It must be high enough that registering many fake providers is expensive, but low enough that honest new providers, who have not yet earned any rewards, are not discouraged from joining.

11.2 Reputation Tolerance

Each task specifies a reputation tolerance T (FR8), and a provider's reputation only decreases when their distance from the final aggregated result exceeds the average distance by more than T (FR37). Without this margin, roughly half of the providers would lose reputation on every request, even when all predictions are close together. T is set per task because an acceptable margin depends on the units and scale of the task's output.

12 Likely Changes

LC1: The set of supported AI inference models may expand beyond the regression model used in the initial proof of concept. The inference processing and aggregation components may need to be adapted to support additional model types.

LC2: The number and behaviour of participating inference providers may change over time as providers join or leave the marketplace. The system's aggregation, reputation, and reward mechanisms may need to accommodate different levels of provider participation.

LC3: The prediction aggregation, reputation, and reward mechanisms may be refined as the proof of concept is evaluated. These changes may be required to improve prediction reliability and address issues such as inaccurate predictions, malicious providers, or collusion.

LC4: User requirements for the marketplace interface and the presentation of inference results may evolve as the system is tested. The interface may need to be updated to improve usability and make prediction quality information easier to understand.

13 Unlikely Changes

LC5: The fundamental purpose of DAIIM is unlikely to change. The system will continue to provide a marketplace for requesting AI inference and obtaining aggregated predictions from participating providers.

LC6: The system is unlikely to move away from its intended web-based form during the scope of this project. Users will continue to access the marketplace through a modern web browser with an internet connection.

LC7: The project is unlikely to introduce a physical hardware component. DAIIM will remain a software-based system that coordinates inference requests, provider predictions, and marketplace operations.

14 Appendices

14.1 Appendix A: Additional Materials

TODO

14.2 Appendix B: Development Plan

[This section is optional. It is used to explain the plan for developing the software. In particular, this section gives a list of the order in which the requirements will be implemented. In the context of a course this is where you can indicate which requirements will be implemented as part of the course, and which will be "faked" as future work. This section can be organized as a prioritized list of requirements, or it could show the requirements that will be implemented for "phase 1", "phase 2", etc. —TPLT]

14.3 Appendix C: Values of Auxiliary Constants

[Show the values of the symbolic parameters introduced in the report. —TPLT]

[The definition of the requirements will likely call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance. —TPLT]

[The value of FRACTION, for the Maintainability NFR would be given here. —TPLT]

- USABILITY_COMPLETION_RATE: the minimum fraction of usability test participants who must complete each task without assistance (NFR2). To be determined.
- MINIMUM_PROVIDER_STAKE: the minimum stake a provider must maintain for an inference task to remain registered for that task (FR43). To be determined.
- MAX_REPUTATION_PENALTY: the maximum reputation decrease a provider can receive for a single inference request (FR38). To be determined.
- SLASH_FRACTION: the fraction of a provider's stake slashed for an invalid prediction or for not submitting a prediction (FR41, FR42). To be determined.
- REPEAT_OFFENSE_COUNT: the number of slashing offences that causes a provider to be unregistered (FR44). To be determined.
- REPEAT_OFFENSE_WINDOW: the number of most recent accepted inference requests over which slashing offences are counted (FR44). To be determined.

14.4 Appendix D: Generative AI Disclosure

This section provides an overview of how generative AI tools were used in the making of this document.

AI Tools Used

- Claude Opus 5.5 (Anthropic), used through Claude Code.

How and Where Used

Claude Opus 5.5 was used throughout this document to help ideate and draft its content. The team provided the tool with the Problem Statement and Goals and the Development Plan, and used it to brainstorm the stakeholders, stakeholder requirements, functional requirements, and non-functional requirements that the document should address, and to produce first drafts of the text. In the Functional Requirements section, for example, it helped split broad descriptions of the marketplace's behaviour into atomic, testable requirements, grouped by the phase of an inference request they belong to, and it suggested symbolic constants, such as SLASH_FRACTION, for values that are not yet decided. No skills or custom agents were created to review this document. It was also used to draft the text of this appendix.

The tool was used as an aid, not as a replacement for the team's own work. Team members were involved in every sentence of the document: each AI-assisted draft was read, discussed, and edited by the team, and ideas that did not fit the project were discarded or rewritten.

Verification of AI-Generated Content

Every sentence in this document was reviewed by at least one team member before it was included. AI-assisted content was checked against the Problem Statement and Goals and the Development Plan to confirm that it describes behaviour the team intends DAIIM to have, that it does not introduce requirements or design decisions the team has not agreed to, and that it is consistent with the technologies the Development Plan lists. Cross-references between requirements were checked by hand, and each requirement was reviewed to confirm that it is atomic, testable, and does not prescribe a design. Values that are still undecided are left as symbolic constants in Appendix C and will be set once the proof of concept has been evaluated. The team remains responsible for the accuracy, completeness, and consistency of this document, regardless of whether material was first drafted by a team member or with the help of an AI tool.

14.5 Appendix E: Reflection

[This isn't part of the SRS, but it is part of the course. —TPLT]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

1. What went well while writing this deliverable?

Nathan: For the SRS and Hazard Analysis documents, we were able to learn a lot about our project by identifying potential hazards and discussing how we could mitigate them. For example, one of our main concerns was determining how the system would handle collusion attacks, which we discussed further during a meeting with our supervising professor. Through ongoing discussions among team members and with our professor, we were able to identify potential hazards and consider appropriate mitigation strategies We took a similar approach when developing the SRS, first identifying our functional requirements with guidance from our professor and ensuring that the different aspects of our project's scope were adequately represented. Overall, maintaining consistent communication within our team and with our professor allowed us to make incremental progress, receive regular feedback, and address uncertainties as they arose. This helped us ensure that our documents accurately reflected our project's intended scope and that we were progressing in the right direction.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Nathan: One pain point our team encountered during this deliverable was determining the appropriate level of detail to include in certain sections. For example, when writing the critical assumptions in the Hazard Analysis, we were unsure whether explicitly stating that the system required at least one non-malicious inference provider would be too restrictive. Making this a formal assumption could imply that the system depended on this condition to operate correctly, potentially requiring additional assumptions or safety measures to address the risks associated with malicious providers. To address this, we decided to treat malicious provider behaviour as a potential hazard within the Failure Mode and Effects Analysis rather than making the presence of a non-malicious provider a concrete assumption. This allowed us to acknowledge and analyze the risk without assuming that malicious behaviour could be ruled out.

   Another pain point was leaving our team review call until the final two days before the deadline. The call lasted over three hours, making it difficult to coordinate everyone's availability and review the document efficiently. For future deliverables, we plan to create pull requests and have our sections ready for review three to four days before the deadline. This will allow team members to review each other's work at times that fit their schedules, provide feedback incrementally, and avoid relying on a single, lengthy meeting close to the deadline.

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

   Aidan: Most of our stakeholder requirements were inspired by speaking to our supervisor, who acts as our client, and by discussions with peers who represent the two kinds of users of DAIIM. Of the 15 stakeholder requirements in the Stakeholder Requirements section, roughly 12 came directly out of these conversations:

   - **Task requesters (SR1 to SR8):** Conversations about who would actually

use a decentralized inference marketplace, such as emergency management agencies forecasting river levels, led us to focus on consensus and reliability. This inspired the requirements that results combine predictions from multiple independent providers (SR3), remain available when some providers are offline (SR4), come with information about their quality (SR5), and are not controlled by a single organization (SR6). Peers without a blockchain background pointed out that requesters should not need to understand decentralized systems to use DAIIM (SR8), and that they should be able to see and verify the payments they authorize (SR7).

- **Inference providers (SR9 to SR12):** Discussions with our supervisor about why anyone would contribute their models to the network shaped all four provider requirements, especially that providers must be able to earn a profit (SR10) and that honest, accurate predictions must be more profitable than dishonest ones (SR11). These later drove most of the staking, reputation, reward, and penalty functional requirements.

The three secondary stakeholder requirements on data privacy, protection, and retention (SR13 to SR15) were inspired less by direct conversations and more by our own research into regulatory expectations and by the hazards identified in the Hazard Analysis. Similarly, most of the detailed functional requirements, such as the exact rules for acceptance periods, reputation updates, and slashing, were derived by the team from the stakeholder requirements rather than requested directly. In this sense, client and peer conversations set the goals of the system, while the team translated those goals into specific, testable behaviour.

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

   Aidan: I will be focusing on the architecture and endpoints of DAIIM's backend server, and three of the software engineering courses I have taken will help with this work.

   - **SFWRENG 2AA4 (Software Design I — Introduction to Software Development):** This course taught me how to break a system into modules with clear responsibilities and well-defined interfaces, and how to apply design principles such as information hiding, low coupling, and high cohesion. These ideas will guide how I separate the backend's concerns, such as handling inference requests, communicating with providers, and submitting transactions to the on-chain programs, so that each part can be developed and tested independently.
   - **SFWRENG 3A04 (Software Design III — Large System Design):** This course covered architectural styles, design patterns, and how the components of a large system interact. It will help me choose an architecture for the backend that fits DAIIM's mix of off-chain and on-chain components, and design endpoints that give the frontend and inference providers a clear, consistent interface to the

system. It also prepared me to document these decisions in the design documents later in the project.

- **SFWRENG 2C03 (Data Structures and Algorithms):** This course taught me how to choose data structures and analyze the efficiency of algorithms. This will help me design backend logic that stays efficient as the number of tasks, requests, and providers grows, such as tracking the phase of each active inference request, matching requests to registered providers, and collecting predictions before aggregation.

Nathan: A course I have taken in the past that will help our team succeed with this capstone project is 3RA3, which focused on software requirements engineering. In this course, we learned about goal modelling and how to distinguish high-level stakeholder goals from concrete software requirements. This knowledge has been useful throughout the capstone project because it has allowed me to better understand what makes a good goal and how to refine it into requirements that describe what our system should accomplish. For example, one of DAIIM's main goals is to provide reliable aggregated inference results. Rather than treating this broad goal as a requirement on its own, we needed to identify the specific system responsibilities that would help achieve it, such as aggregating provider predictions and maintaining provider reputations. Additionally, the knowledge I gained about identifying stakeholders and their goals has helped me consider the different needs of task requesters and inference providers, particularly when designing a marketplace that must balance prediction accuracy with fair provider compensation. Overall, the concepts I learned in 3RA3 have helped me contribute to defining our project's scope and developing requirements that align with its intended goals

Robert:

Keira: Of the courses I have taken throughout my undergraduate degree, I believe the following three will help my team be successful in this capstone project:

- **(Software Design III – Large System Design):** In this course, I learned crucial information about sustainable architectures, methodology of making software architecture decisions, and gained experience designing and developing a complex full-stack program. This included identifying requirements, creating a development plan, and implementing the system. I can use the knowledge and experience gained from this course to navigate the design process, making design decisions, and the implementation of the capstone project.
- **4C03 (Computer Networks and Security):** In this course, I learned about different network protocols, as well as encryption and information security. This knowledge is crucial in addressing some of the hazards we identified and meeting some functional and nonfunctional requirements related to data handling and network transactions.

- **4HC3 (Human Computer Interfaces):** I am currently enrolled in this course and I believe the content related to usability requirement development and designing a user interface that meets the needs of the target user will be helpful, as a huge part of our capstone project is related to the web client through which requesters will interact with the system. Some major nonfunctional requirements which will verify if the project is a success with the target requester user group involve making the technical aspects of the system, such as the blockchain and Web3 wallet infrastructure, accessible and learnable to the average user with limited knowledge in these areas.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

   Much of the knowledge and skills our team will need to collectively acquire to successfully complete this capstone project are related to domain specific knowledge and specific expected technology. This project is highly technical and we will be using tools that we do not have previous experience of implementing.

   Having worked with Dr. Hassan before, Robert was the only member with experience using Solana Devnet and Solana Mainnet to test and implement blockchain transactions and programs. Aidan and Keira had no previous knowledge of or experience with blockchains or smart contracts, and had overcome this learning curve through research. Robert was able to share resources that he had cited previously in his research work, which helped support their learning at the beginning of the project.

   The token economics is another area with which all members do not have or have little experience. As most members do not have experience with Solana, blockchain, or wallets, it naturally follows that experience using SOL tokens on blockchain is something that will have to be acquired for the success of the project.

   None of our four members have implemented an embedded Web3 wallet in our previous projects. Our supervisor, Dr. Hassan, made us aware of the availability of the tool for embedding the wallet in a way that is more familiar to the average user with no experience with Web3 wallets. This will help support our usability non-functional requirements for the users.

   Additionally, a major part of the project is determining a reliable aggregation algorithm for regression. This requires foundational knowledge in theoretical mathematics and precision errors. Malicious providers and collusion attacks also pose a major risk to the system. Understanding methods to combat these risks is necessary for the success of the system and completion of the project.

6. For each of the knowledge areas and skills identified in the previous question, what

are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?